Lambert Solver and MGA Trajectory Optimizer: Implementation Notes and Benchmark Results

Technical Note v1.0

Stavros Kaloumenos
Department of Computer Science and Bioinformatics
University of Thessaly, Lamia, Greece
skaloumenos@uth.gr

9 April 2026

Abstract

This note documents two independent Python tools built from primary literature without external astrodynamics libraries. The first is a Lambert solver implementing Izzo's 2015 algorithm with third-order Householder iteration, validated against textbook solutions and NASA/JPL reference ephemeris data. The second is a differential evolution optimizer for multiple gravity assist (MGA) interplanetary trajectories, augmented with a multilayer perceptron surrogate for fitness pre-screening, and benchmarked against the ESA GTOP Cassini1 problem. Both tools constitute the Python foundation for a planned C++/OpenCL Solar Mission Planner described in a separate research proposal.

Contents

1 Motivation and Scope **2**

2 Lambert Solver **2**
2.1 Algorithm . . . 2
2.2 Ephemeris . . . 2
2.3 Validation Results . . . 3
2.4 Porkchop Plot Results: Earth → Mars 2026 . . . 3

3 MGA Trajectory Optimizer **3**
3.1 Differential Evolution . . . 3
3.2 MGA Fitness Function . . . 4
3.3 Neural Network Surrogate . . . 4

4 Results and Analysis **4**
4.1 Earth → Mars Direct (2D) . . . 4
4.2 Cassini1 — EVVEJS (6D) . . . 5
4.2.1 Gap from the Reference Solution . . . 5
4.2.2 Surrogate Degradation on Cassini1 . . . 6
4.3 Summary Comparison . . . 6

5 Relationship to the Solar Mission Planner **6**

6 Conclusions **7**

1 Motivation and Scope

Interplanetary trajectory design requires solving a sequence of tightly coupled subproblems: determining planetary positions at given epochs, solving Lambert's boundary-value problem to find transfer velocities between those positions, and searching a high-dimensional nonlinear space of departure dates and transfer times for trajectories that minimize propellant expenditure.

The tools described here address each layer independently, then compose them into a working MGA optimizer. The implementation constraint throughout was *no external astrodynamics libraries*: no astropy, no poliastro, no pykep, no pygmo. Every physical calculation is derived from first principles and cited to primary literature. This constraint serves two purposes: it ensures the author understands every line of the computation, and it produces code that can be directly ported to C++ without translation overhead from Python-specific library dependencies.

Both tools are publicly available at https://github.com/mendsergon.

2 Lambert Solver

2.1 Algorithm

The solver implements the algorithm from Izzo [1]. The Lambert boundary-value problem — given position vectors $\mathbf{r}_1$, $\mathbf{r}_2$ and a transfer time Δt, find the departure and arrival velocity vectors — is reformulated in terms of a single free parameter $x \in (-1, \infty)$, where $x < 1$ corresponds to elliptic transfers and $x > 1$ to hyperbolic.

The transfer geometry reduces to a single non-dimensional parameter:

$$\lambda = \pm\sqrt{1 - \frac{c}{s}} \tag{1}$$

where $c = |\mathbf{r}_2 - \mathbf{r}_1|$ is the chord length and $s = (r_1 + r_2 + c)/2$ is the semi-perimeter. The time of flight is non-dimensionalised as $T = \Delta t\sqrt{2\mu/s^3}$.

The equation $T(x) = T_{\text{target}}$ is solved via third-order Householder iteration using analytically computed derivatives:

$$T' = \frac{3Tx - 2 + 2\lambda^3 x/y}{1 - x^2} \tag{2}$$

$$T'' = \frac{3T + 5xT' + 2(1 - \lambda^2)\lambda^3/y^3}{1 - x^2} \tag{3}$$

$$T''' = \frac{7xT'' + 8T' - 6(1 - \lambda^2)\lambda^5 x/y^5}{1 - x^2} \tag{4}$$

This yields quartic convergence; in practice the solver terminates in 3–5 iterations. A Stumpff-function series expansion handles the parabolic singularity at $x = 1$ where the Battin time-of-flight expression is indeterminate.

2.2 Ephemeris

Heliocentric ecliptic (J2000) state vectors are computed from JPL approximate Keplerian elements with linear secular rates [2]. The procedure evaluates mean orbital elements at epoch, solves Kepler's equation via Newton–Raphson, converts to the perifocal frame, and rotates to J2000 ecliptic coordinates via a 3-1-3 Euler sequence (ω, I, Ω). The approximation is valid over 1800–2050 AD, with accuracy ± 1 arcmin for inner planets and ± 15 arcmin for outer planets — more than sufficient for porkchop plot generation, where Δv contours are dominated by transfer geometry rather than sub-arcminute ephemeris errors.

2.3 Validation Results

The solver passes all 13 validation tests. Key results are summarised in Table 1.

Table 1: Lambert solver validation results.

Test	Description	Result
BMW Ex. 5.2	Textbook Lambert (Earth gravity)	Energy/momentum to machine precision
Hohmann recovery	Circular coplanar Earth→Mars	$\Delta v_{\text{dep}} = 2.946$ km/s
Ephemeris check	Earth & Mars at 2026-01-01	Positions ±0.01 AU of JPL Horizons
Energy consistency	$\varepsilon_1 = \varepsilon_2$ at endpoints	Relative error $< 10^{-15}$
Angular momentum	$h_1 = h_2$ at endpoints	Relative error $< 10^{-15}$

2.4 Porkchop Plot Results: Earth → Mars 2026

Table 2 summarises the Earth–Mars 2026 porkchop plot.

Table 2: Earth→Mars 2026 porkchop plot results.

Property	Value
Optimal departure	2026-11-01
Optimal arrival	2027-09-07
Minimum total Δv	5.611 km/s
Departure Δv	3.041 km/s
Arrival Δv	2.570 km/s
Time of flight	310 days
Grid size	$81 \times 112 = 9{,}072$ solves
Compute time	2.8 s
Convergence rate	97.9%

The Hohmann transfer minimum of 5.596 km/s is recovered to within 0.3%. The small discrepancy arises from Mars' orbital eccentricity ($e \approx 0.093$) and inclination, which the Hohmann calculation ignores. The diagonal ridge visible in the contour plot is the 180° transfer singularity — a physically correct feature of the Lambert problem geometry.

3 MGA Trajectory Optimizer

3.1 Differential Evolution

The optimizer implements DE/rand/1/bin [3]. For each candidate $\mathbf{x}_i$ in the population, a trial vector is built as:

$$\mathbf{v} = \mathbf{x}_a + F \cdot (\mathbf{x}_b - \mathbf{x}_c) \quad \text{(mutation)} \tag{5}$$

$$\text{trial}_j = \begin{cases} v_j & \text{if rand()} < CR \\ x_{i,j} & \text{otherwise} \end{cases} \quad \text{(crossover)} \tag{6}$$

$$\mathbf{x}_i \leftarrow \text{trial} \quad \text{if } f(\text{trial}) < f(\mathbf{x}_i) \quad \text{(selection)} \tag{7}$$

where $F = 0.7$ and $CR = 0.9$. Out-of-bound values are reflected rather than clipped, preserving exploration pressure near boundaries.

3.2 MGA Fitness Function

For a planet sequence $[p_0, p_1, \ldots, p_{N-1}]$, the decision vector is $\mathbf{x} = [t_0, T_1, \ldots, T_{N-1}]$ where t_0 is the departure date in MJD2000 and T_k is the transfer time for leg k in days.

At each intermediate planet the flyby cost depends on whether the required turn angle α between $\mathbf{v}_{\infty,\text{in}}$ and $\mathbf{v}_{\infty,\text{out}}$ is geometrically feasible. The maximum achievable deflection at periapsis radius r_p is:

$$\sin\left(\frac{\delta_{\max}}{2}\right) = \frac{1}{1 + r_p v_\infty^2/\mu_{\text{planet}}} \tag{8}$$

The flyby cost is then:

$$\Delta v_{\text{flyby}} = \begin{cases} \big||\mathbf{v}_{\infty,\text{out}}| - |\mathbf{v}_{\infty,\text{in}}|\big| & \text{if } \alpha \le \delta_{\max} \\ \sqrt{v_1^2 + v_2^2 - 2v_1 v_2 \cos(\alpha - \delta_{\max})} & \text{if } \alpha > \delta_{\max} \end{cases} \tag{9}$$

The first case correctly reports zero cost for a geometrically feasible unpowered flyby where the magnitude mismatch can be accommodated. The second computes the minimum impulse at periapsis when the requested turn exceeds the physical limit.

3.3 Neural Network Surrogate

A multilayer perceptron (128, 128, 64 ReLU hidden units, Adam optimizer) is trained on accumulated fitness evaluations and used to pre-screen each generation's trial population. Inputs are normalized to $[0, 1]$ using problem bounds. Fitness values are clipped at 100 km/s before standardization to prevent extreme infeasibility penalties (10^6 km/s) from destroying training gradients. The surrogate is retrained every 10 generations on a rolling reservoir of up to 10,000 observations.

At each generation the full trial population is ranked by surrogate prediction, and only the top 30% trigger a real Lambert solve. The primary evaluation metric is Spearman rank correlation on held-out data rather than MAE — what matters for pre-screening is fitness *ordering*, not absolute accuracy.

4 Results and Analysis

4.1 Earth → Mars Direct (2D)

Table 3 reports 5-run statistics at a budget of 5,000 real evaluations per run.

Table 3: Earth→Mars direct: 5 runs × 5,000 evaluations.

Metric	DE	Surrogate-DE
Best Δv (km/s)	5.608	5.608
Mean Δv (km/s)	5.608	5.608
Std (km/s)	0.000	0.000
Avg evaluations to convergence	1,064	3,335
Avg wall time (s)	0.18	43.62
Reference (km/s)	5.610	

Best mission: departure 2026-10-30, arrival 2027-09-06, time of flight 311 days, total $\Delta v =$ 5.608 km/s. Agreement with the independent porkchop plot minimum (5.611 km/s) is within 0.05%, confirming that the DE optimizer and MGA fitness function are correctly coupled.

Both optimizers find the global minimum consistently across all 5 runs. The surrogate provides no benefit on this problem. DE converges in approximately 1,000 evaluations on a 2D landscape, so there is no evaluation budget to save — the surrogate warmup and retraining overhead make it 240× slower for identical solution quality. This is the expected behaviour: surrogate pre-screening is only beneficial when the real fitness function is expensive relative to surrogate inference cost, and when the budget is the binding constraint. Neither condition holds for a 2D problem.

4.2 Cassini1 — EVVEJS (6D)

Table 4 reports 5-run statistics at a budget of 30,000 real evaluations per run. The ESA GTOP Cassini1 problem encodes the Earth–Venus–Venus–Earth–Jupiter–Saturn sequence with decision vector $\mathbf{x} = [t_0, T_1, T_2, T_3, T_4, T_5]$.

Table 4: Cassini1 EVVEJS: 5 runs × 30,000 evaluations.

Metric	DE	Surrogate-DE
Best Δv (km/s)	9.419	9.422
Mean Δv (km/s)	9.419	13.641
Median Δv (km/s)	9.419	14.219
Worst Δv (km/s)	9.419	20.189
Std (km/s)	0.000	4.001
Avg wall time (s)	27.75	238.10
Reference MGA-1DSM (km/s)	4.930	

Best mission found:

Table 5: Best Cassini1 trajectory found by DE.

Leg	Transfer time	Arrival
Earth → Venus	180 days	1998-05-17
Venus → Venus	415 days	1999-07-06
Venus → Earth	53 days	1999-08-28
Earth → Jupiter	1,057 days	2002-07-20
Jupiter → Saturn	4,634 days	2015-03-28
Total	6,339 days	(17.35 years)

Departure date 1997-11-18 falls within 34 days of the real Cassini launch (1997-10-15), confirming that the problem geometry and ephemeris are correctly implemented.

4.2.1 Gap from the Reference Solution

The 9.419 km/s result is substantially above the 4.930 km/s ESA reference. Two independent factors account for this:

Formulation difference. This implementation uses unpowered MGA only — no deep-space impulses within transfer legs. The ESA best-known solution uses the MGA-1DSM transcription, which allows one deep-space maneuver per leg [4]. The search space here is strictly smaller, so the achievable minimum is higher by construction. This is a deliberate scope choice that keeps the fitness function physically transparent.

Insufficient evaluation budget. The Cassini1 landscape is highly multimodal in 6 dimensions. Published competitive results [6] use evaluation budgets of 100,000–300,000 with population sizes of 100–200. At 30,000 evaluations with population 60, all 5 DE runs converge to the same local

optimum (std = 0.000), indicating stagnation rather than exploration. Increasing the budget is the primary path to improvement within the current formulation.

4.2.2 Surrogate Degradation on Cassini1

The surrogate-assisted DE performs significantly worse than plain DE on Cassini1: mean Δv increases by 44.8% and standard deviation rises from 0.000 to 4.001 km/s. This is a known failure mode of surrogate-assisted optimization on highly multimodal landscapes [5].

The MLP learns a smooth approximation of a jagged fitness surface. Pre-screening on this approximation systematically filters out candidates in unexplored basins that appear unpromising to the surrogate but lead to better optima. The result is reduced population diversity and premature convergence. High variance across runs (9.422 to 20.189 km/s) confirms the surrogate is actively destabilizing the search.

This failure is specific to the combination of high dimensionality, strong multimodality, and insufficient budget. On Earth-Mars the surrogate achieves rank correlation $\rho = 0.996$ on the quadratic test function and matches DE exactly — it can learn smooth landscapes. The diagnostic is the held-out rank correlation during a run: when ρ degrades below a threshold the surrogate is learning noise rather than signal, and pre-screening should be disabled. This adaptive fallback is identified as the primary improvement for the next version.

4.3 Summary Comparison

Table 6 contrasts the two problems across the key dimensions that determine whether surrogate assistance helps or hurts.

Table 6: Problem characteristics and surrogate effectiveness.

Property	Earth→Mars (2D)	Cassini1 (6D)
Dimensions	2	6
Budget binding?	No (DE $\ll$ 5,000 evals)	Yes (stagnates at 30,000)
Landscape	Smooth, unimodal	Highly multimodal
Surrogate rank corr. ρ	0.996 (quadratic proxy)	Degrades during run
Surrogate effect	Neutral (240× slower)	Harmful (+44.8% mean Δv)
Conclusion	Surrogate unnecessary	Surrogate counterproductive

The contrast is instructive: the two problems sit at opposite ends of the landscape-complexity axis, and the surrogate behaviour is predictable from that position. A surrogate is most useful in the intermediate regime — expensive fitness evaluations, moderate dimensionality, landscape smooth enough for an MLP to learn useful ordering but complex enough that naive DE would exhaust its budget before converging.

5 Relationship to the Solar Mission Planner

These tools constitute the validated Python prototype of the computational core described in the Solar Mission Planner research proposal. The mapping is direct:

- The Lambert solver → transfer leg evaluation in the mission planning engine
- The analytical ephemeris → planetary state vector queries (to be replaced by full SPICE/ANISE ephemeris at higher fidelity)
- The DE optimizer → outer optimization loop (to be extended with self-adaptive variants and GPU-parallel population evaluation)

- The surrogate → fitness pre-screening layer (to be made adaptive based on held-out rank correlation monitoring)
- The MGA fitness function → candidate trajectory scoring (to be extended with MGA-1DSM, powered flybys, and aerobraking)

The planned C++ port will replace the Python implementation with OpenCL-accelerated parallel candidate evaluation, enabling population sizes of 500–1,000 at acceptable wall times. The Python codebase serves as the reference implementation against which the C++ port will be validated.

6 Conclusions

Two trajectory optimization tools were implemented from primary literature without external astrodynamics libraries. The Lambert solver achieves quartic convergence and recovers the Earth-Mars Hohmann minimum to within 0.3%. The MGA optimizer correctly solves the Earth-Mars direct problem to within 0.05% of the independent porkchop reference. On the ESA GTOP Cassini1 benchmark the optimizer finds a geometrically plausible trajectory (departure date consistent with the real mission) but stagnates at 9.419 km/s due to insufficient evaluation budget and the absence of deep-space impulses. The surrogate pre-screening improves nothing on simple problems and degrades performance on complex ones — a result that directly motivates adaptive surrogate strategies for the next version.

Both tools are ready for C++ port and serve as the Python foundation for the Solar Mission Planner.

References

[1] D. Izzo, "Revisiting Lambert's problem," *Celestial Mechanics and Dynamical Astronomy*, vol. 121, no. 1, pp. 1–15, 2015.

[2] E. M. Standish, "Approximate Positions of the Planets," JPL Solar System Dynamics, 1992 (updated 2022).

[3] R. Storn and K. Price, "Differential Evolution — A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces," *Journal of Global Optimization*, vol. 11, no. 4, pp. 341–359, 1997.

[4] M. Vasile and P. De Pascale, "Preliminary Design of Multiple Gravity-Assist Trajectories," *Journal of Spacecraft and Rockets*, vol. 43, no. 4, pp. 794–805, 2006.

[5] Y. Jin, "Surrogate-assisted evolutionary computation: Recent advances and future challenges," *Swarm and Evolutionary Computation*, vol. 1, no. 2, pp. 61–70, 2011.

[6] M. Zuo et al., "A case learning-based differential evolution algorithm for global optimization of interplanetary trajectory design," *Applied Soft Computing*, vol. 94, p. 106451, 2020.